CCF 全国青少年信息学奥林匹克联赛

NOIP2026 模拟赛

时间：2026 年 1 月 1 日 09:00 ～ 12:00

题目名称	运筹帷幄	登山	喵了个喵	你的名字
题目类型	传统型	传统型	传统型	传统型
目录	chess	mountain	meow	name
可执行文件名	chess	mountain	meow	name
输入文件名	chess.in	mountain.in	meow.in	name.in
输出文件名	chess.out	mountain.out	meow.out	name.out
每个测试点时限	2.0 秒	1.0 秒	1.0 秒	1.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	20	25	2	25
测试点是否等分	是	是	否	是

提交源程序文件名

对于 C++ 语言	chess.cpp	mountain.cpp	meow.cpp	name.cpp

编译选项

对于 C++ 语言	-O2 -std=c++14 -static

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. `main` 函数的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参考各省的具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈空间内存限制与题目的内存限制一致。
8. 只提供 Linux 格式附加样例文件。
9. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。
10. AK 了请不要大声喧哗，没 AK 也不要。

运筹帷幄（chess）

【题目背景】

棋如人生，落子无悔。步步思量，方能远航。

【题目描述】

zt17 发明了一种独特的益智游戏。在这个游戏中，你将被给予一个棋盘和 n 种棋子。

棋盘可以被看作一个 $H \times W$ 的矩形，上面布有网格，整个棋盘被划分为 HW 个均匀的单位正方形。对于每一个 $1 \le x \le H, 1 \le y \le W$，都有一个坐标为 (x, y) 的方格。

第 i 种棋子有 c_i 枚，而你需要将所有的 n 种棋子放到棋盘上。具体的，对于每一枚棋子，你需要选定一个方格 (x, y) 作为它的位置。允许将多枚棋子放到同一个位置，也允许将不同种类的棋子放到同一个位置。

每种棋子都还有一个限制。具体的，对于一枚第 i 种棋子，它的位置**不能在**以 (x_i, y_i) 为左上角、(x'_i, y'_i) 为右下角的矩形中。形式化的，一枚第 i 种棋子的位置 (x, y) **不能同时满足** $x_i \le x \le x'_i, y_i \le y \le y'_i$。

对于一种填放棋子的方案，它的价值被定义为满足“两枚棋子处于相同方格”这个条件的无序棋子对数量。

我们把每一枚棋子都满足限制的填放棋子方案称作合法方案。zt17 想知道所有合法方案中方案的价值最大可以达到多少。

但他当然不会算啦！于是他想求助你。由于 zt17 数不了太大的数，你只需要输出答案关于 2^{64} 取模后的答案。

【输入格式】

从文件 ***chess.in*** 中读入数据。

输入的第一行包含一个整数 cid，表示测试点编号。样例满足 $cid = 0$。

输入的第二行包含三个整数 n, H, W，含义如题面所述。

接下来的 n 行，每行包含对于一种棋子的描述。第 i 行包含五个整数 $x_i, y_i, x'_i, y'_i, c_i$，含义如题面所述。

【输出格式】

输出到文件 ***chess.out*** 中。

输出一行一个整数，表示合法方案最大价值对 2^{64} 取模后的结果。

【样例 1 输入】

```
0
2 1 2
1 1 1 1 3
1 2 1 2 4
```

【样例 1 输出】

```
9
```

【样例 1 解释】

将 4 枚第 2 种棋子放入方格 $(1,1)$，3 枚第 1 种棋子放入单元格 $(1,2)$，这样的方案价值为 9。可以证明不存在价值更优的方案。

【样例 2 输入】

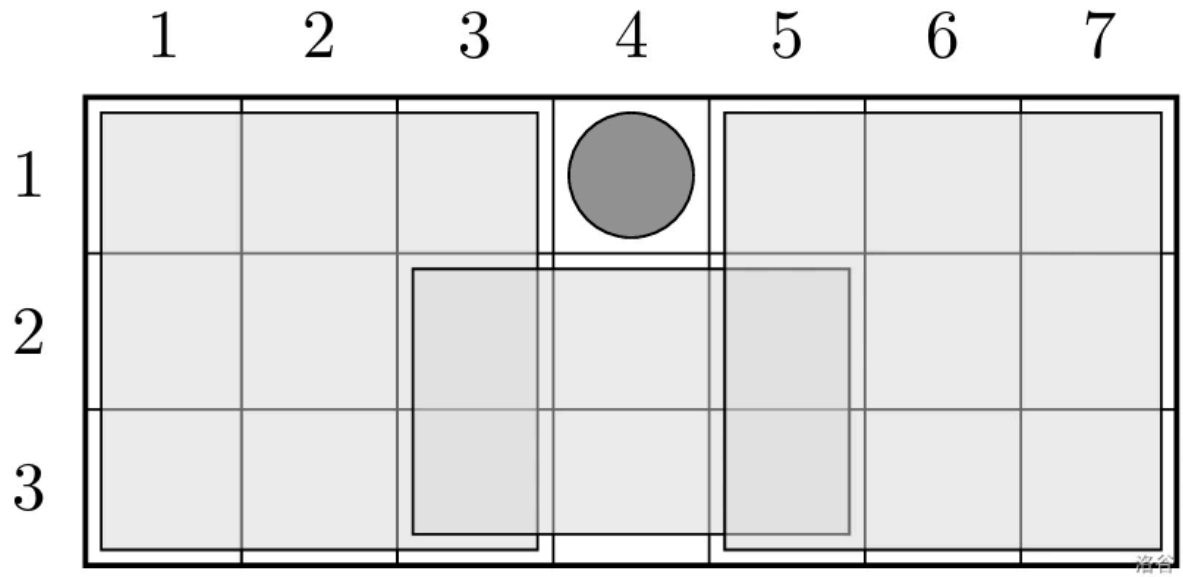
```
0
3 7 3
1 1 3 3 1
5 1 7 3 1
3 2 5 3 1
```

【样例 2 输出】

```
3
```

【样例 2 解释】

限制如下图所示，所有的棋子都可以被放入方格 $(4,1)$。



【样例 3】

见选手目录下的 ***chess/chess3.in*** 与 ***chess/chess3.ans***。
该组样例满足 5 ∼ 8 的测试点约束。

【样例 4】

见选手目录下的 ***chess/chess4.in*** 与 ***chess/chess4.ans***。
该组样例满足 9 ∼ 10 的测试点约束。

【样例 5】

见选手目录下的 ***chess/chess5.in*** 与 ***chess/chess5.ans***。
该组样例满足 11 ∼ 16 的测试点约束。

【样例 6】

见选手目录下的 ***chess/chess6.in*** 与 ***chess/chess6.ans***。
该组样例满足 17 ∼ 20 的测试点约束。

【数据范围】

对于所有的数据，保证：$1 \le n \le 10^5, 1 \le X, Y \le 5000, 1 \le x_i \le x'_i \le H, 1 \le y_i \le y'_i \le W, 1 \le c_i \le 10^4$，且对于每一个 i，至少满足以下条件之一：$x_i \ne 1, x'_i \ne H, y_i \ne 1, y'_i \ne W$。

测试点	n	X，Y	c_i	特殊性质
$1 \sim 4$	≤ 8	≤ 3	≤ 1	无
$5 \sim 8$			$\leq 10^4$	
$9 \sim 10$	$\leq 10^5$	≤ 5000		A
$11 \sim 16$		≤ 500		无
$17 \sim 20$		≤ 5000		

特殊性质 A：对于每一个 i，至少满足以下条件之一：$x_i \neq 1, y_i \neq 1$。

登山（mountain）

【题目描述】

“为什么要攀登？因为山就在那里。”

在 zt17 面前的是一座高度为 n 的山，其中对于每一个高度 $1 \le h \le n$ 都有一个打卡点。

每一个打卡点都有一个路标，它的指向要么朝上要么朝下。当 zt17 在一个打卡点打完卡后，他会遵循路标的指示，朝着路标的指向前进直到下一次进行打卡。具体的，令当前的打卡点高度为 h，如果当前路标写着 `U`，那么下一次打卡的打卡点的高度 h' 必须满足 $h' > h$；如果当前路标写着 `D`，那么下一次打卡的打卡点的高度 h' 必须满足 $h' < h$。

zt17 会从**任意一个**打卡点开始打卡，接着按照路标爬山，在每一个打卡点打卡**恰好一次**。打卡点将按照打卡顺序被记录在集章册上。

由于他要坐缆车回家，所以他十分在意自己最后打卡的打卡点在哪里。他同时也很在意集章册上的打卡顺序会有多少种，于是 zt17 想知道，对于每一个高度 $1 \le h \le n$，总共有多少种可能的打卡顺序，满足最后一个打卡点的高度为 h？

但他当然不会算啦！于是他想求助你。由于 zt17 数不了太大的数，你只需要输出答案关于 $10^9 + 7$ 取模后的答案。

【输入格式】

从文件 ***mountain.in*** 中读入数据。

输入的第一行包含一个整数 cid，表示测试点编号。样例满足 $cid = 0$。

输入的第二行包含一个长度为 n 的字符串，表示所有打卡点的路标类型。该字符串的每个字符要么是 `U`，要么是 `D`。第 i 个字符表示高度为 i 的打卡点类型。

【输出格式】

输出到文件 ***mountain.out*** 中。

输出一行 n 个整数，第 i 个整数代表在高度为 i 的打卡点结束的合法打卡顺序数量对 $10^9 + 7$ 取模后的结果。

【样例 1 输入】

```
0
UUD
```

【样例 1 输出】

```
1 2 1
```

【样例 1 解释】

在第一个样例中，总共有如下 4 种合法打卡顺序：

- $1 \to 2 \to 3$
- $1 \to 3 \to 2$
- $2 \to 3 \to 1$
- $3 \to 1 \to 2$

【样例 2 输入】

```
0
UDUUUD
```

【样例 2 输出】

```
1 2 2 4 8 1
```

【样例 3】

见选手目录下的 ***mountain/mountain3.in*** 与 ***mountain/mountain3.ans***。
该组样例满足 $1 \sim 2$ 的测试点约束。

【样例 4】

见选手目录下的 ***mountain/mountain4.in*** 与 ***mountain/mountain4.ans***。
该组样例满足 $3 \sim 5$ 的测试点约束。

【样例 5】

见选手目录下的 ***mountain/mountain5.in*** 与 ***mountain/mountain5.ans***。
该组样例满足 $6 \sim 9$ 的测试点约束。

【样例 6】

见选手目录下的 ***mountain/mountain6.in*** 与 ***mountain/mountain6.ans***。
该组样例满足 $12 \sim 13$ 的测试点约束。

【样例 7】

见选手目录下的 ***mountain/mountain7.in*** 与 ***mountain/mountain7.ans***。
该组样例满足 $14 \sim 17$ 的测试点约束。

【样例 8】

见选手目录下的 ***mountain/mountain8.in*** 与 ***mountain/mountain8.ans***。
该组样例满足 $18 \sim 25$ 的测试点约束。

【数据范围】

对于所有的数据，保证 $1 \le n \le 5000, s_i \in \{\mathrm{U}, \mathrm{D}\}$。

测试点	n	特殊性质
$1 \sim 2$	≤ 10	无
$3 \sim 5$	≤ 20	
$6 \sim 9$	≤ 500	
10	≤ 5000	A
11		B
$12 \sim 13$		C
$14 \sim 17$		D
$18 \sim 25$		无

特殊性质 A：保证 $s_i = \mathrm{U}(1 \le i < n), s_n = \mathrm{D}$。
特殊性质 B：保证 $s_1 = \mathrm{U}, s_i = \mathrm{D}(1 < i \le n)$。
特殊性质 C：保证 $s_1 = \mathrm{D}$。
特殊性质 D：保证存在一个正整数 k，使得 $1 \le k < n, s_i = \mathrm{U}(1 \le i \le k), s_j = \mathrm{D}(k+1 \le j \le n)$。

喵了个喵（meow）

【题目描述】

游戏“喵了个喵”中有 n 种不同点数的卡牌，依次用 1 到 n 之间的整数表示。定义一套卡牌中，每种卡牌的数量恰好为 2。经过洗牌，这 $2n$ 张纸牌构成了一幅牌堆。**初始牌堆顺序将由特殊方式生成**，详情请见【输入格式】。

在一局游戏中，会给出初始牌堆顺序，和若干个栈。然后玩家将按照牌堆顺序依次取出牌堆中的牌。

对于每张牌，玩家可以进行依次"放入操作"将当前牌放入任意一个栈。在一次放入操作中，玩家需要给出当前牌将放入的栈的编号，编号从 1 开始。

当所有的牌都放置完毕后，玩家需要保证所有栈满足以下条件：可以进行若干次如下的“消除”操作，知道所有牌均被消除完毕：

- 每次操作选择**两个不同的栈**，若这两个栈当前的栈顶卡牌点数相同，则将这两张牌同时从栈顶弹出并消除这两张牌。

现在 zt17 将作为玩家游玩这个游戏。他想知道在一局游戏中栈的数量最小可以达到多少。

但他当然不会算啦！于是他想求助你。他会告诉你 n 和初始牌堆顺序，你需要帮他构造一个编号序列，表示每张牌将要放入的栈的编号。

由于 zt17 不是啥子，所以你不用给出消除的操作序列，他会帮你判断你的构造是否正确。

在每个测试点中，你的得分**将与编号序列的最大值 mx 有关**。详情请见【评分方式】。

【输入格式】

从文件 ***meow.in*** 中读入数据。

输入第一行包含两个整数，分别为 n 和 $seed$。程序需使用这两个参数生成初始牌堆顺序。

我们提供一份 ***sample_meow.cpp***，其中实现了下述函数：

```
std::vector<int> generate_cards(int n,uint64_t seed)
```

将 $n, seed$ 作为参数调用函数会返回给定的牌堆，下标从小到大即为牌堆顺序。你可以直接在该程序基础上实现你的程序。

【输出格式】

输出到文件 ***meow.out*** 中。

输出第一行包含 $2n$ 个整数，表示牌堆从上往下每张牌将被放入的栈的编号。**输出的编号不能大于 10^6，否则你会得到 0 分的判定结果。**

【样例 1 输入】

```
3 17
```

【样例 1 示例输出】

```
1 1 1 2 2 3
```

【样例 1 解释】

实际的牌堆顺序为 $1, 2, 3, 1, 3, 2$。

【数据范围】

对于所有的数据，满足 $1 \le n \le 5 \times 10^5$。

Subtask	分值	n
1	5	≤ 5
2	95	$\le 5 \times 10^5$

保证总测试点数量不超过 30。

【计分方式】

在分值为 p 的 Subtask 中，设 m 表示在当前 Subtask 中 mx 的最大值，则程序将获得 $p \cdot f(m)$ 分，其中 f 的计算方式如下表所示：

m	$f(m)$
$m \le 190$	1
$190 < m \le 260$	$1 - \frac{m-190}{600}$
$260 < m < 1415$	$\max(0.9 - \frac{\sqrt{m}-260}{100}, 0.55)$
$1415 \le m \le 1500$	$0.45 + \frac{1500-m}{850}$
$1500 < m \le 5 \times 10^5$	$0.1 + \frac{\max(700-\sqrt{m},0)}{2000}$
$m = 5 \times 10^5 + 1$	0.1
$5 \times 10^5 + 1 < m \le 10^6$	0.05

【测试程序方式】

我们在下发文件中提供了 ***meow/checker.cpp*** 以及配套编译环它。使用方式为在命令行中用下述指令编译：

```
g++ -o checker checker.cpp
```

运行方式：

```
./checker meow.in meow.out meow.out
```

如果你的输出合法，那么会返回结果 `Accepted!!!!!` 并附有使用的栈编号最大值 mx，否则则会返回结果 `Unaccepted...` 并附有栈中剩余卡牌信息。

你的名字（name）

【题目描述】

Alice 和 Bob 正在玩一个游戏。游戏规则如下：

- 有一些怪物，每个怪物都有一个指定的生命值（HP）。
- Alice 和 Bob 轮流行动，Alice 先手。
- 在 Alice 的回合，他选择一个怪物并将其 HP 增加 1。
- 在 Bob 的回合，她选择一个怪物并将其 HP 减少 2。如果怪物的 HP 变为 0 或更低，该怪物就会消失。
- 当有 k 个怪物消失时，游戏结束。

Alice 的目标是尽可能晚地结束游戏，而 Bob 的目标是尽可能早地结束游戏。最初没有怪物。你需要处理 Q 次操作，操作类型如下：

- 1 X Y：将 HP 为 X 的怪物数量变为 Y。
- 2 K：假设双方都采取最优策略，询问如果游戏以当前怪物状态开始且 $k = K$ 时，Bob 需要多少回合才能结束游戏。

注意，游戏并非实际发生，怪物也不会真的消失。

【输入格式】

从文件 ***name.in*** 中读入数据。

输入的第一行包含两个整数 cid, Q，分别表示测试点编号和操作次数。样例满足 $cid = 0$。

接下来的 Q 行第一个整数 T_i 表示操作类型，如果 $T_i = 1$ 则接下来有两个整数 X_i Y_i，如果 $T_i = 2$ 则接下来有一个整数 K_i。

【输出格式】

对于每一个 $T_i = 2$ 的操作，输出一行答案。

【样例 1 输入】

```
0 6
1 1 4
2 3
1 2 3
2 6
1 2 2
2 6
```

【样例 1 输出】

```
3
7
8
```

【样例 2 输入】

```
0 20
1 1 12
2 12
1 2 15
2 12
2 3
1 12 10
2 27
1 14 6
2 7
2 43
2 22
1 8 7
1 1 11
2 49
1 5 19
2 38
2 8
1 12 14
1 16 1
2 24
```

【样例 2 输出】

```
12
12
3
42
7
```

```
246
25
301
91
8
32
```

【样例 3】

见选手目录下的 ***name/name3.in*** 与 ***name/name3.ans***。
该组样例满足 1 ∼ 2 的测试点约束。

【样例 4】

见选手目录下的 ***name/name4.in*** 与 ***name/name4.ans***。
该组样例满足 3 ∼ 6 的测试点约束。

【样例 5】

见选手目录下的 ***name/name5.in*** 与 ***name/name5.ans***。
该组样例满足 7 ∼ 8 的测试点约束。

【样例 6】

见选手目录下的 ***name/name6.in*** 与 ***name/name6.ans***。
该组样例满足 9 ∼ 11 的测试点约束。

【样例 7】

见选手目录下的 ***name/name7.in*** 与 ***name/name7.ans***。
该组样例满足 12 ∼ 14 的测试点约束。

【样例 8】

见选手目录下的 ***name/name8.in*** 与 ***name/name8.ans***。
该组样例满足 15 ∼ 18 的测试点约束。

【样例 9】

见选手目录下的 ***name/name9.in*** 与 ***name/name9.ans***。
该组样例满足 19 ∼ 22 的测试点约束。

【样例 10】

见选手目录下的 ***name/name10.in*** 与 ***name/name10.ans***。
该组样例满足 $23 \sim 25$ 的测试点约束。

【数据范围】

对于所有的数据，设当前怪物总数为 S，保证 $1 \le Q \le 3 \times 10^5, 1 \le T_i \le 2, 1 \le X_i \le 10^6, 0 \le Y_i \le 10^6, 1 \le K_i \le S$，至少存在一个查询操作。

测试点	Q	X_i	S	特殊性质
$1 \sim 2$	≤ 500	≤ 500	≤ 500	无
$3 \sim 6$	$\le 10^5$	$\le 10^6$	$\le 10^6$	A
$7 \sim 8$	$\le 3 \times 10^5$	≤ 1		无
$9 \sim 11$		≤ 2		
$12 \sim 14$	≤ 1000	≤ 1000		
$15 \sim 18$	$\le 3 \times 10^5$	$\le 10^6$		B
$19 \sim 22$				C
$23 \sim 25$				无

特殊性质 A：保证 $T_i = 1 (1 \le i < Q), T_Q = 2$。
特殊性质 B：保证 $X_i \ne 1$；
特殊性质 C：保证存在一个整数 k，使得 $K_i = k$。